

Chromium as a tool of logistical power: A material political economy of open-source

Bolun Zhang and Davide Carpano

Big Data & Society
January–June: 1–12
© The Author(s) 2023
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/20539517231182399
journals.sagepub.com/home/bds



Abstract

Open-source software is used by almost all technology companies and has become an integral part of the technical infrastructure of digital capitalism. Generally, developers of open-source software have been viewed as a social movement at odds with the capitalist profit motive. This idealized view has been challenged as companies have made significant investments in open-source since the early 2000s. Current research frames corporate participation in open-source as fundamentally extractive in nature, failing to account for these sizable investments. Through a historical analysis of Google's Chromium browser project, we provide another way to understand corporate participation in open-source. This article takes a material political economic approach to argue that control of open-source projects can grant companies logistical power that enables them to influence standards and shape the Internet as an infrastructure for digital capitalism.

Keywords

Open-source software, material political economy, standard setting, internet governance, infrastructure, data capitalism

Introduction

Free and open-source software (FOSS) constitutes the backbone of the digital economy. The majority of mobile devices that people use to access the Internet run on Android, an open-source operating system (OS) based on Linux; the server those mobile devices communicate with as they browse the Web is also likely also running on Linux; and the website being visited is probably hosted either by Apache or Nginx, two open-source web server projects that together account for 50%–65% of the server market (Netcraft, 2022). FOSS is so pervasive and provides such significant value to the digital economy that it has been described as the “dark matter of the Internet” (Greenstein and Nagle, 2014).

FOSS is both a process and a product, and, by using open source licenses, allows anyone to view, modify, and redistribute a program's code. This differentiates it from proprietary software, which hides a program's source code from users. Beyond the code it produces, FOSS can be understood as a model for software development in which collaboration between geographically dispersed individuals takes place online via email and on code hosting sites like GitHub. Most projects are collaboratively developed in a meritocratic process resembling a lively bazaar in which anyone is free to contribute, with the best ideas rising to the top (Raymond, 1999). In scholarly and popular depictions, FOSS developers are often romanticized as a group of heroic individuals

motivated by the intrinsic value of a technical challenge (Sennett, 2009; Söderberg, 2008). Their approach to software development has been understood as a challenge to major industry powers and has contributed to the public good by creating the conditions for a startup renaissance (Eghbal, 2016).

Starting in the early 2000s, large corporations began participating in several important open-source projects, leading many to ask whether FOSS developers might be unwitting participants in their own exploitation (Barron, 2013; Moore and Taylor, 2009). This view, informed by the idea that the aim of corporate participation in FOSS is the extraction of free labor (Terranova, 2000), has failed to account for sizable investments companies make in open-source software. This is best illustrated by the Linux OS, arguably the most influential FOSS project. In 2017, 45.9% of the changes to the Linux project came from the top ten corporate contributors alone, with less than 10% coming from unaffiliated individuals (Corbet and Kroah-Hartman, 2017).

Department of Sociology, University of California San Diego, La Jolla, USA
Both authors contributed equally to this article.

Corresponding author:

Davide Carpano, Department of Sociology, University of California San Diego, La Jolla, USA.

Email: dcarpano@ucsd.edu



Companies are not only participating in existing projects; they have also adopted a strategy of open sourcing internally developed software. Google has been a pioneer in this space, releasing major projects like Android, Chromium OS, and Chromium browser as open-source software. Srnicek (2017) has suggested that corporations like Google internally develop FOSS projects as part of a broader strategy of attracting users to a platform where they can be monetized; however, this explanation does not account for corporations' use of permissive licenses, which allow competitors to appropriate their code and release competing versions of the same program.

We address this gap through a historical analysis of Chromium, an open-source browser project started by Google in 2008. Building from Donald MacKenzie's (2017, 2021) concept of material political economy, we provide a third way to understand corporate involvement in FOSS: Control over major open-source projects can grant companies the logistical power to set de facto standards. By logistical power, we are referring to "the ability to mobilize the natural world for political effect" (Mukerji, 2010: 402), which transforms its possessor into an "obligatory passage point for those wanting to exercise modern forms of power" (Joyce and Mukerji, 2017: 2). Taking a capacious view of politics as the shaping and ordering of the world through actions and discourse, we show how, through Chromium, Google gains the logistical power to shape the Internet (Law and Mol, 2008). We explore how Chromium acts as an obligatory passage point for the standardization of Web-based application programming interfaces (API). APIs, a defined set of instructions that enable two programs to communicate, are important sites of governmentality (Bucher, 2013; Dijck et al., 2018). Control over these interfaces has enabled companies like Google to shape the Internet as an infrastructure for data capitalism.

This paper begins with a detailed review of the social science literature on FOSS before discussing methods and data. Next, we provide a chronology of Chromium's development and introduce the vision of the Web that Google advances through the project. In this section, we analyze WebUSB API to demonstrate how the de facto standards set by Chromium reinforce Google's view of the web. In the subsequent section we analyze a collection of APIs developed by Google known as WebExtension. The adoption of these APIs by all major browsers demonstrates how de facto standards can become coercive, forcing other actors to accept them as formal standards even when no consensus exists. Finally, we compare Chromium to other browsers, and argue that its success in shaping standards is the result of it being open-source as well as the specific manner in which Google has integrated it within a broader ecosystem of products and services. We conclude by summarizing our contributions and identifying avenues for future research on the subject.

Understanding corporate involvement in FOSS

What is now commonly referred to as open-source software began as "free software", a movement launched by Richard Stallman in the mid-1980s. By emphasizing the social value that could come from sharing code, the free software movement provided a non-market impetus for producing high-quality software (Landini, 2012). In the late 1990s the free software movement experienced a schism wherein a subset of participants began calling the software they produced 'open-source' in an attempt to moderate the overtly political nature of the movement by focusing on the technical merits of FOSS. Despite this shift in emphasis, free and open source software are seen as two sides of the same coin. Within the social science literature, there is a sense that the FOSS movement has produced software that is not only technically but also morally superior: "[T]hese methods are also the best in a larger social, moral, and political sense.[...] Open-source projects represent an idealized vision of democracy in action—run by volunteers and organized only on an informal, ad hoc basis; decisions are made by consensus rather than decree" (Ensmenger, 2004).

This understanding of FOSS is supported by two key assumptions. First, FOSS is understood to be rooted in liberal values such as free speech, equality of opportunity, and meritocracy (Coleman, 2013). FOSS is a critical social practice that challenged the IP regime undergirding the software industry in the late 1990s (Coleman, 2013), and spurred a successive phase of capitalist accumulation in the digital economy (Barron, 2013). Second, the FOSS movement is "as much about process as it is product" (Ensmenger, 2004: 104), and is primarily concerned with creating and maintaining "the technical, legal, practical, and conceptual means to its own existence as a public" (Kelty, 2008: 3). Accordingly, FOSS developers are depicted as "programming enthusiasts, committed to maximizing access to computer equipment and software, and to the information necessary to improve these tools and innovate with them" (Barron, 2013: 598). Based on this framework, corporate participation in FOSS is seen as an aberration that is actively resisted by developers (Birkinbine, 2015).

The overarching understanding of corporate participation in FOSS development assumes that by reducing the value of intellectual property, FOSS has forced companies to find ways to profit from control and exploitation of reputational capital (Barron, 2013). There are two ways in which the literature has characterized this exploitation. One approach understands corporate involvement as an exemplification of the value that free labor provides in the digital economy (Terranova, 2000). In this view, corporate involvement is seen as a ploy to extract free labor from both willing and unwilling subjects (Lund and Zukerfeld, 2020;

Velkova, 2016). A second approach frames FOSS as bait that companies use to lure in users who are then monetized in other ways (Fourcade and Klutetz, 2020; Srnicek, 2017; Zuboff, 2019). While these two approaches are not mutually exclusive and do account for many cases of corporate involvement, neither is able to explain cases where companies open-source internally developed software.

Chromium as a case study is particularly instructive, as data released in 2019 highlighted that 92% of commits—the process of saving a change to a project’s repository—made to the project came from the 80% of contributors employed directly by Google (Chromium, 2019b). The same data showed that less than 1% of commits came from individuals unaffiliated with a company. If Google’s aim was to attract free labor, they have not done a good job of it. The “bait thesis” also struggles to explain cases like Chromium. While users are indirectly monetized—both from commissions in the application store and by collecting users’ data—such an arrangement does not require the code to be open-sourced. In fact, in the case of Chromium, only a small minority of users use an open-source version of the browser, with the vast majority using Chrome, Google’s free-to-use proprietary implementation.

The emphasis placed on the direct economic benefits that companies derive from FOSS leaves a key question unanswered: Why do companies choose to release software under permissive licenses that allow competitors to copy (or to put it more technically, fork) their code? This question suggests the need to provide a third way of understanding corporate contributions to open-source which looks beyond firms’ immediate financial motives.

Material political economy: Methods and data

To make sense of corporate investment in FOSS, we build from Donald Mackenzie’s framework for analyzing socio-technical systems that he names *material political economy*. Developed in his research on high-frequency trading, material political economy is an approach to the study of technical artifacts that combines an STS emphasis on materiality with a political-economic focus on struggles between firms and the influence of politics (MacKenzie, 2017: 173–174). Empirically, this approach entails “a focus on how the material world is ordered, on the possibility of alternative orderings, and on the often political nature of the issue of which of these orderings becomes real, and on the economic consequences of that” (MacKenzie, 2021: 234). In our case study, we apply the concept to open-source software and, by specifying how the materiality of FOSS converges with political economic issues, we reframe a driving question of the social study of technology: *Do artifacts have politics* (Winner, 1980)?

Our case study of Chromium is historically grounded and draws from the methodology outlined by Denton et al. (2021) in their research on the norms and values embedded in ImageNet, a machine learning data set. The method, rooted in Foucault’s genealogical method, locates the emergence of a set of material conditions in the “emergence, formation, and transformation of practices, discourses, and concepts within a given historical context” (Denton et al., 2021). Accordingly, we begin our analysis by locating the ideologies, discourses, and economic motives that undergird the Chromium project. We then analyze APIs produced by Chromium and show how these ideologies and discourses are enacted materially, demonstrating how economic considerations structure the political struggles over standards within the World Wide Web Consortium (W3C), a key Internet standard-setting organization.

Data

Our analysis proceeds in two stages. First, we use primary and secondary sources to construct our historical analysis of Chromium. We created a database of documents from a variety of sources: (a) Chromium project blog posts, bug report forums, advertisements and technical documentation, and chromium conference proceedings and speech transcripts; (b) blog posts and speeches by major browser or web developers; (c) Google company documents such as SEC filings, marketing material, and white papers. In the first part of our analysis, we argue that Google takes a radical implementation-first approach to standards, producing and disseminating APIs that advance a view of the web as a development platform and web pages as applications.

In the second part, we trace how Google’s implementation-first approach shapes the production of W3C standards. Accordingly, we work from three bodies of data: (a) W3C reports and meeting notes, and discussion forums from relevant working groups; (b) technical documentation and blog posts from the three major browser projects: Chromium, WebKit, Firefox; (c) Caniuse.com: an open data set which tracks browser support for hundreds of APIs (Deveria, 2022).

Reconfiguring the internet: Web pages as applications

The rise of the Internet in the 1980s and 1990s was facilitated by a collection of standard-setting organizations (SSO) whose challenge to traditional forms of standard setting laid the foundation for how the infrastructure would be governed. Viewing the traditional standard setting process as centralized and overly bureaucratic, Internet SSOs opted to take an implementation-first approach which aims to establish ‘rough-consensus’ over a standard before it is formally defined (Russell, 2014;

Yates, 2019). Conceptualizing the Internet as a public resource to be protected from enclosure (Mozilla, 2013), Internet SSOs assume a bazaar-like form, conducting their work in the open and encouraging anyone to join. For example, the W3C has no formal membership or voting procedures, participation in working groups is fully open and carried out online, and anybody is free to join meetings and view the archives of past discussions.

In contrast to the ethos of early web pioneers, Google developed a different view of the Internet. Although Google started as a search engine with the mission to “organize the world’s information”, by the mid-2000s the company had developed a suite of free-to-use web applications that sought to replace commonly used desktop programs. Applications like Drive, Docs, and Maps showed the potential of web applications to challenge their desktop counterparts, but remained audacious projects that were severely limited by existing browser technology, demanding access to hardware that existing API standards could not provide (Kennedy, 2008).

Under the premise that “most of what we use the web for on a daily day-to-day basis aren’t just web pages, they’re applications”, Chromium was developed with the goal of optimizing the performance of web apps (Google, 2008). Two innovations—a modern JavaScript engine and a multi-process architecture—were particularly important for realizing this vision. A JavaScript engine is what allows a browser to execute code written in the JavaScript programming language (the language of choice for web apps). Chromium’s engine, called V8, featured a cutting-edge design that utilized a ‘just-in-time’ compiling method, a process in which code is dynamically compiled during execution rather than before. In addition to V8, Google’s introduction of a multi-process architecture improved the performance of web applications by allowing them to be compiled independently of other browser processes. Separating web pages and applications into independent processes not only resulted in faster load times, but also prevented issues in one process from causing the entire browser to crash (a common problem for browsers in the early- to mid-2000s).

Framing web pages as application environments was more than a statement about technical capacity. The APIs that web applications utilize act as a site of governmentality over the Internet by controlling flows of data, defining ecosystems, and regulating the pace of innovation (Bucher, 2013; Dijck et al., 2018). As much as the technical benefits of rich web apps appear to empower users, in the context of digital capitalism, there is an implicit imbalance between the gift giver and receiver (Fourcade and Klutetz, 2020). The powerful websites that JavaScript enables have become critical sites of data extraction, with clear benefits to companies like Google. Compared to other tracking technologies like browser cookies (blocks of data stored on a user’s browser by websites), which can only track basic device information and browser history, JavaScript

applications allow detailed event-based tracking of a user’s online behavior.

Given the potential to unlock new flows of data, Google’s interest in shaping emerging standards had clear benefits to the company: “With a richer set of APIs we can build more interesting apps allowing people to do more online. The more people do online, the more they can use our services” (Google, 2008). In the following section, we detail how Google leverages the Chromium project to shape de facto standards with the goal of enabling more powerful web applications.

Chromium as an API production line

Since the project’s inception, Google has maintained that the decision to open-source Chromium was rooted in a desire “to help drive the web forward” (Google, 2008). To achieve this, Google adopted a radical implementation-first approach towards standards. Data on API support among browsers highlights that Chromium supports 65 APIs that Firefox does not implement, compared to 22 APIs adopted only by Firefox (Deveria, 2022). The disparity is even more stark when considering unofficial standards and working drafts², with Chromium being alone in supporting 52 unofficial and working draft APIs, compared to Firefox’s 11. (Table 1)

Not only does Chromium implement a massive amount of unofficial standards, it also provides a forum to specify these standards that largely bypasses the W3C (Developers, 2018). The Capabilities project, launched by Google in 2018, is emblematic of this approach, with the cross-industry collaboration described as “an effort to close gaps in the web’s capabilities enabling new classes of applications to run on the web” (Developers, 2018). Commonly referred to by its codename, Fugu, the initiative is firmly rooted in a view of the web as an application development and distribution technology. Although Fugu has no formal relationship with official standard-setting bodies, its practices resemble those of Internet SSOs, taking place in the open with no formal membership.

The most detailed explanation of the theoretical underpinnings of Fugu can be found in a blog post by Alex Russell, a former Chrome developer now employed by Microsoft: “the web thrives or declines to the extent it can accomplish the lion’s share of the things we expect most computers to do” (Russell, 2022). Russell’s argument in favor of Fugu is premised on a theory known as “platform adjacency”, which understands the Web and technologies like JavaScript as ‘meta-platforms’ which thrive on their ability to act like a computer. Because meta-platforms sit above the OS, the failure to develop new APIs capable of accounting for new use cases would result in a relevance gap. Seen from this framework, “web apps should be able to do anything native iOS/Android/desktop apps can,”

Table 1. Application programming interfaces supported by Firefox and Chromium (source: caniuse.com, accessed on 12 February 2022).

Category	Supported by both	Not supported by both	Supported by Chromium but not by Firefox	Supported by Firefox but not by Chromium	Total
Candidate recommendation	66	9	6	6	85
Working draft	83	31	25	6	145
Unofficial	27	30	27	5	89
Other	59	11	7	7	84
All	235	81	65	22	403

making the continual addition of new capabilities to the Web necessary for it to remain relevant (Developers, 2018).

Guided by this view, Fugu provides a platform for Google’s implementation-first strategy, allowing Chromium to rapidly produce and adopt new standards. The result is that Google gains influence over the standard-setting process and reinforces Chromium’s market advantage. In an interview with us, a former Mozilla engineer recalled that “Google proposed new API standards so quickly that Mozilla could hardly follow, and this diverted the already limited development resources that Mozilla has” (Anonymous, 2022). While Mozilla is a relatively small organization, Google’s strategy has created similar problems for major players like Microsoft. It was the inability to retain compatibility with Chromium that led Microsoft to drop its own browser engine in 2019, instead opting to build their Edge browser from the Chromium base (Warren, 2019b). While this strategy creates problems for other browser developers, Google’s primary intent is to expand browser capabilities in order to facilitate the creation of powerful web applications.

WebUSB and De Facto standards

Among the many APIs that are maintained under the auspices of Fugu, there are similar APIs that aim to grant browsers access to hardware devices, bridging the web with the “physical world” (WICG, 2022). Included in this group are WebUSB, WebSerial, and Web Bluetooth APIs which, as their names suggest, grant browsers access to USB, serial, and bluetooth devices. In this section, we focus on WebUSB to show how Chromium overcomes criticism from other browser vendors to produce de facto standards that expand browser capabilities.

WebUSB is a simple but powerful API that grants browsers access to non-standard USB devices, allowing hardware vendors to develop browser-based software development kits (SDK; software tools used by developers to create programs on specific platforms). While most USB devices communicate with the OS through one of over a dozen standard device classes, many hobbyist devices (like Arduinos) do not implement a standard class. In the past, USB devices such as these could not be accessed by

a browser, and the SDKs that accompanied them were desktop applications. From hardware manufacturers’ perspectives, WebUSB enables them to write a single web-based SDK rather than developing and maintaining distinct desktop applications for two or three different OSs (Martin, 2017).

Despite being integrated into all the major Chromium-based browsers and achieving widespread use by developers, WebUSB remains controversial, with both Firefox and WebKit refusing to implement the API, citing privacy and security concerns. Shortly after its release, Mozilla marked the API as “harmful,” noting that the security risks of “exposing USB devices to the Web are too broad to risk exposing users to them or to properly explain to end users to obtain meaningful informed consent” (Mozilla, 2022). WebKit’s developers took a similar stance, listing the API among several others that would not be implemented due to “security, and other concerns” (WebKit, 2022). Without the support of the major browser vendors, WebUSB has failed to enter the W3C’s official standard track. Instead, it remains a working draft submitted to the Web Platform Incubator Group (WPICG)—a lightweight venue where ideas for potential standards can be centralized and discussed prior to being placed on the official standard track.

Staying true to the implementation-first strategy, Google proceeded to implement WebUSB as a standard feature in Chromium before discussions surrounding security concerns had been addressed. Within the WPICG, some developers used the issue tracker to point out their concerns, with one developer arguing that “WebUSB is, in theory, a good idea[...] However, in practice[...] USB devices are not designed to be ‘connected’ through a remotely web content-controlled API on a hostile network” (wolfbeast, 2016). Despite a clear lack of consensus regarding potential security risks involved with the APIs, one of the Google employees leading the WebUSB WPICG marked the issue as closed. When pressed to explain why he had closed the issue, the engineer simply stated that “it is too broad to be actionable” (reillyeon, 2017).

Several months after the issue was closed, security issues forcefully reemerged. The controversy began in 2018, when a report demonstrating how hackers could exploit WebUSB to gain access to user accounts was published by two

security researchers. In a relatively simple hack, a convincing fishing website could leverage the API to query USB-based two-factor authentication devices, allowing hackers to unlock a user's account after tricking them into typing their username and password (Greenberg, 2018). In the issue tracker a user expressed frustration at how quickly the security issue had been closed: "By closing this issue, you have suppressed discussion on this topic and reduced this issue's visibility. By rushing this standard into production without adequate consideration of security aspects you and other contributors have exposed millions of users to a new class of attack" (jaseg, 2018). To developers like this one, the report not only validated their security concerns, it called aspects of the standard setting process into question.

Even amidst criticism of Google's decision to embrace WebUSB, the API rapidly became a de facto standard following its implementation. The success of WebUSB, which is a standard feature on nearly 80% of desktop browsers (Deveria, 2022; MDN, 2022), is largely attributable to Chromium's dominant market share. The speed with which these standards proliferate is evident in the more recently released Web Serial, one of WebUSB's sister APIs; released in February 2021, the API is already a default feature on 75% of browsers.

Firefox and WebKit have maintained their position, refusing to implement WebUSB and similar APIs, drawing the ire of Chromium developers. In one instance, a prominent Chromium developer derided Mozilla and WebKit as "the committee to cast the web in amber", equating their refusal to adopt new APIs to a "structural under-investment" that would spell disaster for the Internet: "Every organism cast in amber is, after all, deceased" (Russell, 2022). By refusing to go along with WebUSB, the developers of these browsers are choosing to risk losing users to Chromium-based browsers rather than ceding standard-setting authority to Chromium.

As more developers and hardware vendors make use of WebUSB, users will expect browsers to implement the API, and the cost of this resistance to browser developers will inevitably increase. In this sense, WebUSB can be thought of as a standard-in-waiting. While it may be a de facto standard, WebUSB has failed to achieve universal adoption and is, therefore, unable to make the leap from being a tool of local control to one of global regulation (Galloway, 2004). The ability to exert control outside of one's own ecosystem, according to Galloway (2004) is gained when an API becomes a formal standard. This is the case with the APIs we examine in the following section, which shows that Chromium is not only a tool to set de facto standards but also a site where Google achieves the global control that comes through standard setting, without having to conduct their work within the formal channels of Internet SSOs.

Standardizing browser extensions

Of the three major open-source browser projects, two have refused to implement WebUSB. This refusal may suggest that competition on the Internet promotes choice. We refute this narrative through an analysis of the WebExtension APIs, the framework that defines how APIs related to browser extensions work. In this section, we show how de facto standards can become coercive, forcing others to converge around particular technical designs and shaping standards.

The convergence of all major browsers around WebExtension is particularly salient because browser extensions are a fundamental part of the modern web experience, granting users control over their online interactions. In what follows, we trace the emergence of WebExtension and discuss how its success forced Firefox to abandon its legacy extension APIs. We then discuss a series of controversial changes made by Google to the WebExtension framework, underlining the manner in which other browser vendors were forced to accept these changes and illustrating the extent of Google's logistical power over standards.

Becoming the prototype

Prior to Chromium's release, Firefox was on track to overtake Internet Explorer (IE) as the most-used browser. Firefox's embrace of an open Internet and features like a large extension ecosystem—based on the XUL and XPCOM standards—were major selling points. Rather than adopting these standards, Chromium took a different approach and developed an extension API designed to optimize the browser's multi-process architecture and utilize its new JavaScript engine. In 2015, Mozilla decided to abandon Firefox's API framework in favor of Chromium's, a decision taken for two interrelated reasons.

First, Mozilla had been working on a new multi-process architecture for Firefox, and the impending switch would require their legacy extension APIs to be upgraded. WebExtension had numerous advantages over Firefox's legacy APIs, including native support for multi-process browsers, an architecture with which the XUL/XPCOM framework was not compatible (Mozilla, 2015, 2017, 2019a). The transition to WebExtension, finalized in 2016, was a costly undertaking and resulted in a loss of flexibility for developers, with many complaining that the new APIs lacked key features offered by the XUL/XPCOM framework.

While the reason behind adopting WebExtension was technical in nature, the decision to adopt a more restrictive extension framework based on Chromium's extension APIs was also the result of market forces. Chromium's dominant market share, together with its ability to connect users, developers, and new technical infrastructures, combined

to make its extension APIs a gateway to a set of infrastructural resources like extensions. To gain access to these resources, Mozilla had little choice but to base WebExtension on Chromium's extension framework: "To deny that Chrome had a very large market share and a modern, supportable extension ecosystem would have been foolish" (Mozilla, 2018). Given the choice between going it alone, or embracing Chromium's extension framework, Mozilla—conscious of its limited resources and need to maintain compatibility with Chromium's extension ecosystem—chose the latter.

Firefox was not the last browser to convert to Google's extension model. Microsoft adopted Chromium's extension architecture in 2016, several years prior to transitioning its Edge browser to the Chromium base. Safari, the last major browser to adopt WebExtension, announced that it too would adopt WebExtension at its developer conference in 2020. With all three major browser projects converging around Chromium's extension framework, WebExtension has become the de facto standard governing the entire extension ecosystem.

What made this convergence even more significant was the failure of a bottom-up attempt to set formal standards at the W3C. In 2016, the Browser Extension Community Group (BECG) was formed with the intent to standardize extension APIs. Although Mozilla partnered with Microsoft and Opera to work in this group, the BECG lacked support and engagement from the broader community, especially from Google (BECG, 2017; Frivoal, 2021; Mozilla, 2018). By the end of 2017, it had become clear that a de facto standard already existed, rendering the BECG's work redundant at best and irrelevant at worst, and the BECG ceased operations. At the BECG's last teleconference, its chairman lamented: "Everyone had their own extension mechanisms, and then people started looking to use Google's one, and then Opera switched to blink and got it ... Firefox and MS have things that are very close. This CG [community group] was trying to standardise [sic] it ... documentation and code went well, not much standardisation work" (BECG, 2017).

With Chromium's extension APIs emerging as the de facto standard, Google had gained logistical power over the entire extension ecosystem and ignored existing attempts to set formal standards. This power became evident in a series of controversial changes to the WebExtension APIs proposed in 2018, which other browser makers were eventually forced to adopt.

Controversial changes

In November 2018, the Chromium project published a beta version of Manifest v3 (MV3), a set of proposed changes to WebExtension. Almost immediately, two changes, which many developers read as an attempt to shackle extensions, became sites of controversy. The manner in which Google

has been able to unilaterally implement these changes highlights the extent to which APIs are sites of governmentality, and reveals how Google leverages logistic power to shape the material world.

The first change that MV3 introduced involved replacing background pages with service workers. While service workers perform the same task previously done by background pages, which provide an environment wherein an extension can run separately from a web page or tab, it does so by taking a different approach. Background pages were popular among extension developers both because they would run until the extension went idle and granted access to a web page's programming interface, or direct object module (DOM), which allows extensions to directly alter its content. Service workers, by contrast, restrict access to the DOM, and run on a timer that shuts down the process whether or not the extension is idle.

This change would affect countless extensions, with one developer noting that extensions based on service workers would offer "degraded capabilities" (Chromium, 2019a). Another explained how the changes would affect their extension: "We use our background script's webpage in order to copy/paste plain and styled text to the clipboard in our extension. [...] If this background page DOM went away, I am not sure how we would be able to recreate that functionality" (Chromium, 2019a). While the impacts of the switch would be felt across the entire extension ecosystem, the second controversial change appeared to target a smaller subset of extensions.

The part of MV3 that drew the most ire revolved around replacing the WebRequest blocking API with DeclarativeNetRequest. The proposal was seen by many as an intentional attack on ad-blocking and anti-tracking extensions (O'Flaherty, 2022), a claim that Google has disputed (Warren, 2019a). WebRequest had been the cornerstone for these extensions, and, unlike DeclarativeNetRequest, placed significant control in the hands of extension developers. The key difference between the two APIs, both of which enable extensions to alter content on a web page, had to do with how the blocking requests were processed. Under WebExtension, extensions were able to receive, modify, and block requests sent from a server to the browser, enabling them to modify the server's response (in order to remove an ad, or pornographic content, for example). DeclarativeNetRequest would take away this ability, with blocking requests being handled by the browser rather than the extension itself.

Under the new framework, extension developers would pre-register lists of rules with the browser and cede the ability to directly modify web pages. Besides placing a seemingly arbitrary limit to the number of rules, which started out at 30,000 before gradually increasing, rules could no longer be updated on the fly and would require the end-user to update an extension before new rules could be applied. Developers of blocking extensions expressed frustration at

the possibility of losing the functionality allowed by WebRequest, and argued in vain that the proposed changes would make their products significantly worse at their jobs (AdGuard, 2019; Ad Remover et al., 2019).

Coercive standards

As the controversy around MV3 unfolded with few obvious solutions emerging, a new attempt to standardize extension APIs was initiated within the W3C. In 2021, all of the major browser developers, including Google, announced the formation of the WebExtension Community Group (WECG). Although Google helped form the WECG, and has one of their engineers serving as co-chair (alongside an Apple employee), the company chose to press ahead and implement MV3 before a formal standard could be agreed upon.

The argument for adopting the changes put forward by Google revolved around claims that MV3 would enhance user privacy while improving browser performance and security (Chromium, 2019c). Both assertions were contested within the WECG, with some pointing out that giving service workers a 5-min lifespan would actually reduce efficiency, only to see further discussion tabled by Google representatives (WECG 2021a, 2021b, 2022a). When it came to privacy, developers of ad-blocking and anti-tracking extensions emphasized that the changes would prevent them from reacting to new online threats, compromising users by exposing them to harmful trackers (WECG, 2022b). Unanimous in their denunciation of MV3, these developers saw the loss of control over how extensions could modify browser behavior as “an attack on the user’s liberty” (WECG, 2021b) that would “decrease the level of user agency in Chromium” (Chromium, 2019a).³

Shortly after the WECG’s inaugural meeting in September of 2021, and before any major issues could be resolved, Google announced a firm timeline for the transition to MV3 on Chromium. The API would be phased in starting in January of 2022, and by June of 2023 any extension not implementing MV3 would stop running on Chromium. At subsequent meetings, extension maintainers expressed frustration at the timeline, seeing it as impossible to meet. At the March 2022 WECG meeting, one developer lamented that he had “never seen a transition like this in my 20 years [as a developer]” (WECG, 2022c). Not only would they be forced to rewrite entire extensions, but they would have to do so without many of the features in MV2 that enabled their core functions in the first place.

Responding to pushback, Google representatives deflected criticism by resurrecting the argument that hesitance to adopt new standards amounted to wishing to ‘cast the web in amber’: “as with many changes in MV3, we are paving the road for the future, even if their use is not clear now” (WECG, 2022d). As Google pressed ahead, the WECG’s efforts to resolve the most controversial

aspects of MV3 stalled. The implementation-first approach taken by Google has prevented an alternative from arising, forcing others to either accept Chromium’s implementation or forfeit access to its user base and technical resources. Although the WECG group continues to meet regularly, it had always been obvious that Mozilla and Apple would have to accept Chromium’s implementation of the APIs: “In the absence of a true standard for browser extensions, maintaining compatibility with Chrome is important for Firefox developers and users” (Mozilla, 2019b).

With frustration mounting, one developer went as far as to question the value of the WECG: “What about the vast majority of developers not represented in these meetings? Will their extensions just stop working one day because of Google’s poor communication?” (WECG, 2022c). At the following meeting, Google’s representative validated this sentiment, claiming that Chromium had been pursuing a new version of WebExtension since “before this WECG group was founded[...] and were already in the process of going down this path before we have had broader discussions with other browser vendors and the community” (WECG, 2022d). Although they claimed that Chromium wished to collaborate with others to realize this vision, they remained adamant that they would not “abandon the initial drive and motivations that led us down this path” (WECG, 2022d).

In the 2 years since its founding, the WECG’s efforts to define a common standard for extension APIs have produced little in the way of tangible results. Recognizing this, and knowing that Chromium would soon only accept extensions that conformed to MV3, Mozilla was eventually forced to take a position on which elements of MV3 it would adopt. Rather than choosing a side, they decided to implement both the DeclarativeNetRequest API and the service worker framework, while simultaneously maintaining support for WebRequest and background pages. Much like Mozilla’s initial switch to WebExtension in 2015, the decision to adopt MV3 in its entirety was motivated by the need to remain compatible with Chromium’s extension ecosystem: “Support for MV3, by virtue of the combined share of Chromium-based browsers, will be a de facto standard for browser extensions in the foreseeable future” (Wu, 2022).

The controversy surrounding extension APIs is a powerful example of the indirect relationship between economic interest and technical design. For many years, Google has highlighted ad-blocking and anti-tracking extensions as significant threats to the company’s revenues, noting that “such technologies could adversely affect our operating results” in communications with investors (Alphabet Inc., 2018). Yet, Google did not ban these applications from the Chrome Web Store, a move that would likely have led to public outrage. Instead, Google leveraged its logistical power to push MV3, and reshape the topology of the Internet in a much more subtle way.

Accounting for Chromium's success

In the previous section, we showed how Chromium produces *de facto* standards that shape the Internet according to Google's vision. Here, we address the question of why Google so successfully wields logistical power relative to other browser vendors. Through a comparison with Microsoft's IE and two open-source browsers (Firefox and WebKit), we show how this logistical power is the result of the manner in which Google has integrated an open-source strategy within an ecosystem of products and services where the company has achieved market dominance.

Without an open-source strategy, market dominance alone does not translate into logistical power over the standard setting. In order to become universally applicable, standards and protocols must be transparent, which contradicts proprietary software's closed approach. As Galloway (2004: 171) notes, "while closed-source technologies appear to be protological due to their often monopolistic position in the marketplace, a true protocol can not be closed or proprietary." Microsoft's failure to convert IE's market dominance into logistical power over standard setting highlights this contradiction.

First released in 1995, IE quickly became the dominant Web browser, holding 80% of the market by 1998. Microsoft's dominance over the browser market was largely due to the company's decision to release IE for free, bundling it with the Windows OS (a strategy that resulted in a landmark antitrust lawsuit against the company). Initially, it appeared that IE market dominance would allow it to play a significant role in shaping formal standards. By the early 2000s, IE's widespread use meant that many early web standards, such as Cascading Style Sheets (a computer language, widely considered to be a cornerstone of the Web, used for describing the presentation of documents written in HTML or XML) and the basic structure of web applications, were created to be compatible with the browser (W3C Wiki, 2019).

While resulting in substantial market share and influence over Web standards initially, the decision to bundle IE with Windows backfired over time. As rapidly iterating standards began to move away from IE's proprietary APIs, the browser struggled to adapt to the constant change. Microsoft had deliberately blurred the boundaries between the OS and the browser to the point where they shared key components and the same update mechanisms (Baldwin, 2021). This made it difficult for IE to implement rapidly iterating standards, as IE's updates were tied to the Windows update mechanism which made regularly updating the browser a challenge. Additionally, any changes made to the browser could affect Windows' overall performance. Ultimately, after a brief honeymoon period, IE became a target of criticism, with major figures like Tim Berners-Lee openly criticizing Microsoft's sluggish implementation of W3C standards (Svensson, 2008).

The lack of an open-source strategy for IE and the struggle to remain standard-compliant resulted in performance issues which contributed to a rapid decline in the browser's market share throughout the mid-2000s. IE's loss was Firefox's gain as the project, established in 2004, managed to achieve 30% market share by 2008 and appeared set to overtake IE in a matter of time. This changed after Chromium's release; by 2010 Chromium's market share surged to over 50%, with Firefox's share declining in lock-step.

In contrast to Microsoft's proprietary strategy, Google followed in Firefox's footsteps, and released Chromium as an open-source project. By doing this, Google's aim was to position Chromium as a public good that would benefit all browsers, since developers could "take what they want" from this project without any obligation to pay Google or even to contribute fixes back to the Chromium base (Google and McCloud, 2008). Chromium's success relative to Firefox in influencing standards can be attributed both to the browser's early technical advantages and its incorporation into Google's massive ecosystem.

Chromium may be an open-source project, but the way that Google approached its development resulted in key technological advantages over its competitors. This was most evident in the secretive manner in which the V8 JavaScript engine and Chromium's multi-process architecture were developed. The original maintainer of SpiderMonkey, Firefox's just-in-time JavaScript engine, recalled learning about V8 in 2006 and being kept in the dark about it because of Google's secrecy (Eich, 2011). The development and release of Chromium's multi-process architecture followed a similar path. Initially, Google based its browser engine on WebKit, an open-source browser project maintained by Apple. Rather than contribute changes made to the code base back to the WebKit main branch, Google decided early on that it would maintain a separate Chromium tree, declining to contribute its multi-process support back to WebKit when asked. This left the WebKit team to develop a multi-process architecture from scratch, which was later released as WebKit2 (Hacker News, 2013).

Google's guarded approach to open-source notwithstanding, its success relative to Firefox and WebKit is also attributable to the specific way that the company has combined an open-source strategy with the large ecosystem of products and services that it controls. This ecosystem benefits Chromium in a variety of intentional and unintentional ways and contributes to the browser's dominant market share. Unintentional advantages come in the form of tight integration between Chromium and Google services, which pushes users away from competing browsers. This is evident in the case of YouTube, which stopped supporting IE in 2009, a decision made by a group of Google engineers frustrated by IE's poor implementation of W3C

standards (Zacharias, 2019). The optimization of Google services to run best on Chromium has drawn condemnation from a former Mozilla vice president, who noted that Google products often performed poorly on Firefox, with glitches often taking weeks to resolve (Nightingale, 2019). Whether or not these glitches were accidental, as Google claims, they ultimately led to a slow but steady exodus of users from Firefox to a more “reliable” browser.

Google also leverages its ecosystem in more obvious ways. The most striking example is their bundling of Chromium with Android, the Google-controlled open-source mobile phone OS that powers 80% of the world’s smartphones. While this strategy recalls Microsoft’s bundling approach, Google lacks the same control that Microsoft is able to exert over the proprietary Windows OS. Instead, Google controls this ecosystem through additional layers of licensing and certification. As a condition to license Google’s Play service (the official “app store”), Google required phone makers to agree to pre-install Chrome alongside other Google applications. This practice, which contributed to Chrome’s initial surge in popularity, was eventually restricted in the European Union following successful antitrust action by the European Commission in 2017 (Commission, 2016).

Conclusion

By situating Chromium as the site in which users, developers, and technical architectures meet, we show how open-source acts as a tool of logistical power. Our paper illustrates the co-productive relationship between political-economic struggles over the shape of the Internet and its materiality. As the case of WebExtension makes clear, it is through Chromium that Google translates its technical expertise into logistical power over the standard setting. In this way, Google mobilizes the material world to shape political possibilities by creating a “second nature” (Mukerji, 2010) in which the company can thrive. The result has been the creation of a technical infrastructure that enables the proliferation of increasingly powerful web applications. This infrastructure, to which Chromium acts as an obligatory passage point, has been vital for the expansion of data collection and circulation that has fueled digital capitalism (Sadowski, 2019).

Our case sheds light on how we might conceptualize power within the digital economy and supports a view of the Internet as a distributed network with multiple competing centers of power in which standards and protocols are a key site of governmentality and control (Galloway, 2004; Mathew, 2016). While Google is not the only company capable of wielding logistical power through control of open-source projects, it is one of a few organizations with the resources to maintain the ecosystem required to unlock open-source’s potential. As we showed in the previous section comparing Chromium with IE and Firefox,

Google’s ability to use open-source as a tool of logistical power is rooted in the incorporation of open-source software into a broader ecosystem. This suggests that critical sites of struggle, such as infrastructural chokepoints, should be focal point in studies that draw from the material political economy framework.

MacKenzie’s goal with the material political economy was to preserve Actor-Network Theory’s attentiveness to the material world while overcoming its struggle to fully account for power (Law, 1999; MacKenzie, 2021). Future studies that build from this framework might consider investigating how companies collectively wield logistical power through open-source foundations in large community-based projects like Linux. A similar approach might also be taken to the study of other technologies and their infrastructures, such as Artificial Intelligence, and the tension between open and proprietary strategies to its development that companies like Meta and OpenAI are actively negotiating.

Rather than approaching the Internet through a politics of nostalgia that seeks a return to a mythical time when the Internet was decentralized, looking backward should allow us to imagine what other futures might have been (Tarnoff, 2022). When approached from this perspective, it becomes clear that we must not wait for a Machiavellian network builder to emerge and solve the problems that plague the Internet. Instead, we urge a return to the form of political discussion envisioned by Hannah Arendt in *The Human Condition*—where citizens gather to dispute their differences in acknowledgment that it is from disagreement that novelty emerges, and invention is made possible. But that day will only come after we have devised new forms of Internet governance capable of accounting for concentrations of logistical power.

Acknowledgments

The authors would like to thank the members of UC San Diego’s Institute for Practical Ethics Data Analytic Working Group, where an early version of this paper was presented. We are also grateful to Lilly Irani and Dorothy Howard for reviewing drafts. Finally, we would like to thank the anonymous reviewers whose generous and insightful comments improved this article.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) received no financial support for the research, authorship, and/or publication of this article.

ORCID iD

Davide Carpano  <https://orcid.org/0000-0002-0936-4797>

Note

1. The first FOSS license to be developed, the General Public License (GPL) required any software using its code to be entirely licensed under the same terms. Another class of “permissive” licenses only require the original code and direct modifications to it to be licensed under the same terms, otherwise software utilizing this code can be published using a different, even proprietary, license. (2) Working drafts are endorsed by the W3C, but are not on an official standard track. (3) According to the W3C, a user-agent is “any software that retrieves, renders and facilitates end user interaction with Web content, or whose user interface is implemented using Web technologies.”

References

- AdGuard (2019) Our comment on Google Chrome’s proposed extensions platform change. *AdGuard Blog*. <https://adguard.com/en/blog/our-comment-on-chrome-ad-blocking.html>.
- Ad Remover, Adblock, Adblock Plus, et al. (2019) Feedback from the ad blocking community on the proposed changes to webRequest and declarativeNetRequest. *Google Docs*. https://docs.google.com/document/d/1sKZFojq_fUusrebKsyNHfRk_9kpbIALFPS1Nvpddfs8/edit?usp=embed_facebook.
- Alphabet Inc (2018) FORM 10-K-001-37580.
- Anonymous (2022) On the standards setting of Mozilla.
- Baldwin CY (2021) Design Rules Volume 2: Chapter 17. 3482515, {{SSRN Scholarly Paper}}. Rochester, NY. DOI: 10.2139/ssrn.3482515.
- Barron A (2013) Free software production as critical social practice. *Economy and Society* 42(4): 597–625.
- BECCG (2017) Browser Extension TPAC breakout session 08 Nov 2017 Browser Extension Community Group. <https://browserext.github.io/minutes/2017-11-08.html>.
- Birkinbine BJ (2015) Conflict in the commons: Towards a political economy of corporate involvement in free and open source software. *The Political Economy of Communication* 2(2): 3–19.
- Bucher T (2013) Objects of intense feeling: The case of the Twitter API. *Computational Culture* 3.
- Chromium (2019a) 896897—Extensions: Implement Manifest V3—chromium. <https://bugs.chromium.org/p/chromium/issues/detail?id=896897#c23>.
- Chromium (2019b) Intent to Explain: Demystifying the Blink Shipping Process. *Chromium Blog*. <https://blog.chromium.org/2019/11/intent-to-explain-demystifying-blink.html>.
- Chromium (2019c) Web Request and Declarative Net Request: Explaining the impact on Extensions in Manifest V3. *Chromium Blog*. <https://blog.chromium.org/2019/06/web-request-and-declarative-net-request.html>.
- Coleman G (2013) *Coding Freedom: The Ethics and Aesthetics of Hacking*. Princeton, NJ: Princeton University Press.
- Commission TE (2016) Antitrust: Commission informs Google of Android concerns. *European Commission—European Commission*. Text. https://ec.europa.eu/commission/presscorner/detail/en/IP_16_1492.
- Corbet J and Kroah-Hartman G (2017) *Linux Kernel Development Report*. The Linux Foundation.
- Denton E, Hanna A, Amironesei R, et al. (2021) On the genealogy of machine learning datasets: A critical history of ImageNet. *Big Data & Society* 8(2): SAGE Publications Ltd: 1–14.
- Developers C (2018) Unlocking new capabilities for the web. *Chrome Developers*. <https://developer.chrome.com/blog/capabilities/>.
- Deveria A (2022) Caniuse. <https://github.com/Fyrd/caniuse>.
- Dijck Jv, Poell T and Waal Md (2018) *The Platform Society: Public Values in a Connective World*. New York: Oxford University Press.
- Eghbal N (2016) *Roads and bridges: The Unseen Labor Behind Our Digital Infrastructure*.
- Eich B (2011) New JavaScript Engine Module Owner Brendan Eich. <https://brendaneich.com/2011/06/new-javascript-engine-module-owner/>.
- Ensmenger NL (2004) Open source’s lessons for historians. *IEEE Annals of the History of Computing* 26(4): 102–104.
- Fourcade M and Klutzz DN (2020) A Maussian bargain: Accumulation by gift in the digital economy. *Big Data & Society* 7(1): SAGE Publications Ltd: 1–16.
- frivoal (2021) What to do with the older CG? Issue #1 W3c/webextensions GitHub. <https://github.com/w3c/webextensions/issues/1>.
- Galloway AR (2004) *Protocol: How Control Exists After Decentralization*. Cambridge, Mass.: MIT Press.
- Google (2008) Welcome to Chromium. *Chromium Blog*. https://blog.chromium.org/2008/09/welcome-to-chromium_02.html.
- Google and McCloud S (2008) Google on Google Chrome—comic book. <http://blogoscoped.com/google-chrome/>.
- Greenberg A (2018) Chrome’s WebUSB Feature Leaves Some Yubikeys Vulnerable to Attack|WIRED. <https://www.wired.com/story/chrome-yubikey-phishing-webusb/>.
- Greenstein S and Nagle F (2014) Digital dark matter and the economic contribution of Apache. *Research Policy* 43(4): 623–631.
- Hacker News (2013) Blink: A rendering engine for the Chromium project | Hacker News. *Hacker News*. <https://news.ycombinator.com/item?id=5489025>.
- jaseg (2018) Security Risks ahead. Issue #50 WICG/webusb. *GitHub*. <https://github.com/WICG/webusb/issues/50>.
- Joyce P and Mukerji C (2017) The state of things: State history and theory reconfigured. *Theory and Society* 46(1): 1–19.
- Kelty CM (2008) *Two Bits: The Cultural Significance of Free Software*. Durham, NC: Duke University Press.
- Kennedy N (2008) The story behind Google Chrome. *Niall Kennedy*. <https://www.niallkennedy.com/blog/2008/09/google-chrome.html>.
- Landini F (2012) Technology, property rights and organizational diversity in the software industry. *Structural Change and Economic Dynamics* 23(2): 137–150.
- Law J (1999) After ANT: Complexity, naming and topology. *The Sociological Review* 47(1_suppl): SAGE Publications Ltd: 1–14.
- Law J and Mol A (2008) Globalisation in practice: On the politics of boiling pigswill. *Geoforum; Journal of Physical, Human, and Regional Geosciences* 39(1): Environmental Economic Geography: 133–143.
- Lund A and Zukerfeld M (2020) *Corporate Capitalism’s Use of Openness: Profit for Free?* Cham, Switzerland: Springer Nature.
- MacKenzie D (2017) A material political economy: Automated trading desk and price prediction in high-frequency trading. *Social Studies of Science* 47(2): SAGE Publications Ltd: 172–194.
- MacKenzie D (2021) *Trading at the Speed of Light: How Ultrafast Algorithms Are Transforming Financial Markets*. Princeton, N.J.: Princeton University Press.

- Martin E (2017) WebUSB - Arduino Project Hub. <https://create.arduino.cc/projecthub/edwin-martin/webusb-85b6bb>.
- Mathew A (2016) The myth of the decentralised internet. *Internet Policy Review* 5(3): 1–16.
- MDN (2022) Web Bluetooth API - Web APIs. https://developer.mozilla.org/en-US/docs/Web/API/Web_Bluetooth_API.
- Moore P and Taylor PA (2009) Exploitation of the self in community-based software production: Workers' freedoms or firm foundations? *Capital & Class* 33(1): SAGE Publications Ltd: 99–119.
- Mozilla (2013) The Mozilla Manifesto. *Mozilla*. <https://www.mozilla.org/en-US/about/manifesto/>.
- Mozilla (2015) The Future of Developing Firefox Add-ons. *Mozilla Add-ons Community Blog*. <https://blog.mozilla.org/addons/2015/08/21/the-future-of-developing-firefox-add-ons/>.
- Mozilla (2017) Firefox 57.0, See All New Features, Updates and Fixes. *Mozilla*. <https://www.mozilla.org/en-US/firefox/57.0/releasesnotes/>.
- Mozilla (2018) WebExtensions/FAQ—MozillaWiki. <https://wiki.mozilla.org/WebExtensions/FAQ>.
- Mozilla (2022) Mozilla Specification Positions—WebUSB. <https://mozilla.github.io/standards-positions/#webusb>.
- Mozilla (2019a) Comparison with XUL/XPCOM extensions. *Firefox Extension Workshop*. <https://extensionworkshop.com/documentation/develop/comparison-with-xul-xpcom-extensions/>.
- Mozilla (2019b) Mozilla's Manifest v3 FAQ. *Mozilla Add-ons Community Blog*. <https://blog.mozilla.org/addons/2019/09/03/mozillas-manifest-v3-faq/>.
- Mukerji C (2010) The territorial state as a figured world of power: Strategies, logistics, and impersonal rule. *Sociological Theory* 28(4): 402–424.
- Netcraft (2022) July 2022 Web Server Survey. *Netcraft News*. <https://news.netcraft.com/archives/2022/07/28/july-2022-web-server-survey.html>.
- Nightingale J (2019) Johnathan Nightingale on Twitter: "When I started at Mozilla in 2007 *archive.ph*. <https://archive.ph/tgIH9>.
- O'Flaherty K (2022) Firefox Just Gave 3 Billion Google Chrome Users A Reason To Switch. *Forbes*. <https://www.forbes.com/sites/kateoflahertyuk/2022/06/13/firefox-just-gave-3-billion-google-chrome-users-a-reason-to-switch/>.
- Raymond E (1999) The cathedral and the bazaar. *Knowledge, Technology & Policy* 12(3): 23–49.
- reillyyeon (2017) Security Risks ahead. Issue #50 WICG/webusb. *GitHub*. <https://github.com/WICG/webusb/issues/50>.
- Russell A (2022) Platform Adjacency Theory. *Infrequently Noted*. <https://infrequently.org/2020/06/platform-adjacency-theory/>.
- Russell AL (2014) *Open Standards and the Digital Age*. New York, NY: Cambridge University Press.
- Sadowski J (2019) When data is capital: Datafication, accumulation, and extraction. *Big Data & Society* 6(1): SAGE Publications Ltd: 1–12.
- Sennett R (2009) *The Craftsman*. London, England: Penguin UK.
- Söderberg J (2008) *Hacking Capitalism: The Free and Open Source Software Movement*. New York, NY: Routledge.
- Smicek N (2017) *Platform Capitalism*. Malden, MA ; Oxford: Polity.
- Svensson P (2008) Creator of Web spots a flaw in Internet Explorer. <https://web.archive.org/web/20080913220037/http://ap.google.com/article/ALEqM5ijt9c0Dlxfdf3QocKPEhKUVzXIGAD9343KEO1>.
- Tarnoff B (2022) *Internet for the People: The Fight for Our Digital Future*. London, England: Verso Books.
- Terranova T (2000) Free labor: Producing culture for the digital economy. *Social Text* 18(2): 33–58.
- Velkova J (2016) Open cultural production and the online gift economy: The case of Blender. *First Monday*. DOI: 10.5210/fm.v21i10.6944.
- W3C Wiki (2019) The history of the Web. https://www.w3.org/wiki/The_history_of_the_Web.
- Warren T (2019a) Google says it's not trying to stop ad blocking on Chrome. *The Verge*. <https://www.theverge.com/2019/2/20/18232972/google-chrome-ad-blocking-manifest-3-changes-response>.
- Warren T (2019b) Inside Microsoft's surprise decision to work with Google on its Edge browser. *The Verge*. <https://www.theverge.com/2019/5/6/18527550/microsoft-chromium-edge-google-history-collaboration>.
- WebKit (2022) Tracking Prevention in WebKit/WebKit. <https://webkit.org/tracking-prevention/>.
- WECG (2021a) WECG Meetings 2021, Public Notes, December 9. https://github.com/w3c/webextensions/blob/6e270877c5c6074bae04a1f4aba46ade489609be/_minutes/2021-12-09-wecg.md.
- WECG (2021b) WECG Meetings 2021, Public Notes, November 11. https://github.com/w3c/webextensions/blob/6e270877c5c6074bae04a1f4aba46ade489609be/_minutes/2021-11-11-wecg.md.
- WECG (2022a) WECG Meetings 2022, Public Notes, February 3. https://github.com/w3c/webextensions/blob/6e270877c5c6074bae04a1f4aba46ade489609be/_minutes/2022-02-03-wecg.md.
- WECG (2022b) WECG Meetings 2022, Public Notes, March 17. https://github.com/w3c/webextensions/blob/6e270877c5c6074bae04a1f4aba46ade489609be/_minutes/2022-03-17-wecg.md.
- WECG (2022c) WECG Meetings 2022, Public Notes, March 3. https://github.com/w3c/webextensions/blob/6e270877c5c6074bae04a1f4aba46ade489609be/_minutes/2022-03-03-wecg.md.
- WECG (2022d) WECG Meetings 2022, Public Notes, March 31. https://github.com/w3c/webextensions/blob/6e270877c5c6074bae04a1f4aba46ade489609be/_minutes/2022-03-31-wecg.md.
- WICG (2022) Web Serial API. <https://wicg.github.io/serial/>.
- Winner L (1980) Do artifacts have politics? *Daedalus* 109(1): 121–136.
- wolfbeast (2016) *Security Risks ahead. · Issue #50 · WICG/webusb*. GitHub. <https://github.com/WICG/webusb/issues/50>.
- Wu R (2022) Manifest v3 in Firefox: Recap & Next Steps. *Mozilla Add-ons Community Blog*. <https://blog.mozilla.org/addons/2022/05/18/manifest-v3-in-firefox-recap-next-steps/>.
- Yates J (2019) Values, media, and genres for standardization. In: Aspray W (eds) *Historical Studies in Computing, Information, and Society: Insights from the Flatiron Lectures. History of Computing*. Cham: Springer International Publishing, 51–82. DOI: 10.1007/978-3-030-18955-6_4.
- Zacharias C (2019) A Conspiracy To Kill IE6 Chris Zacharias. *Chris Zacharias on Svbtte*. <https://blog.chriszacharias.com/a-conspiracy-to-kill-ie6>.
- Zuboff S (2019) *The Age of Surveillance Capitalism: The Fight for a Human Future at the Frontier of Power*. New York, NY: PublicAffairs.